Unconstrained Variable Oracles for Faster Numeric Static Analyses

Vincenzo Arceri
University of Parma

Greta Dolcetti
University of Parma & Ca' Foscari University of Venice

Enea Zaffanella
University of Parma

Context

- Abstract interpretation process may introduce inefficiencies

- Canonical solutions
  - Choose different abstract domain
  - Choose different abstract operators

Abstract Compilation

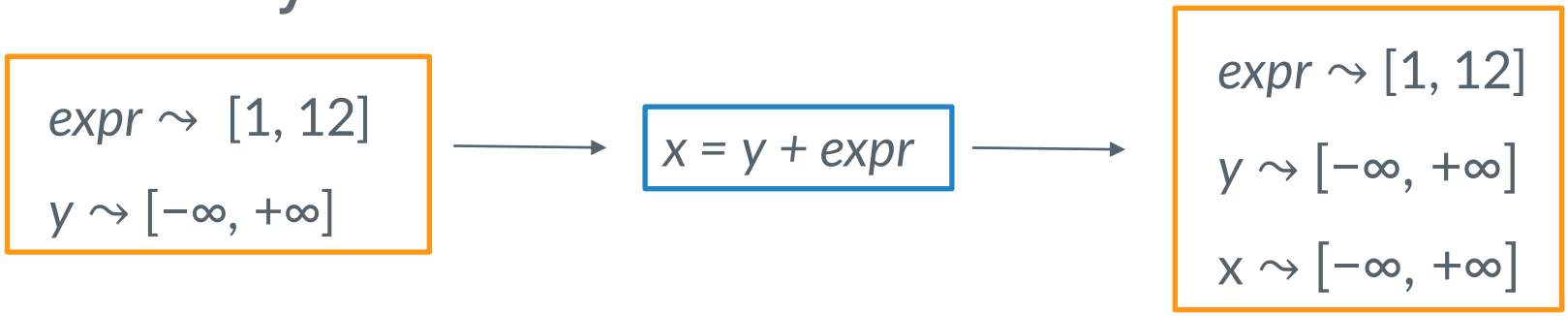
- Abstract Compilation techniques consist of rewriting the intermediate program representation adopted by the analyzer

- Goal: optimize abstract execution
- Trade-off between accuracy and efficiency

Our Approach

- The efficiency of the evaluation process can be improved by propagating known information; improvements may also be obtained by propagating **unknown** information

Likely Unconstrained Variables



- *y* is likely unconstrained -> *x* is likely unconstrained
- If no information is known about *y*, then it is likely that little or no information will be known about the whole expression
- There is little incentive in providing an accurate (and maybe expensive) abstract evaluation for the whole expression

Propagation Of LU Information

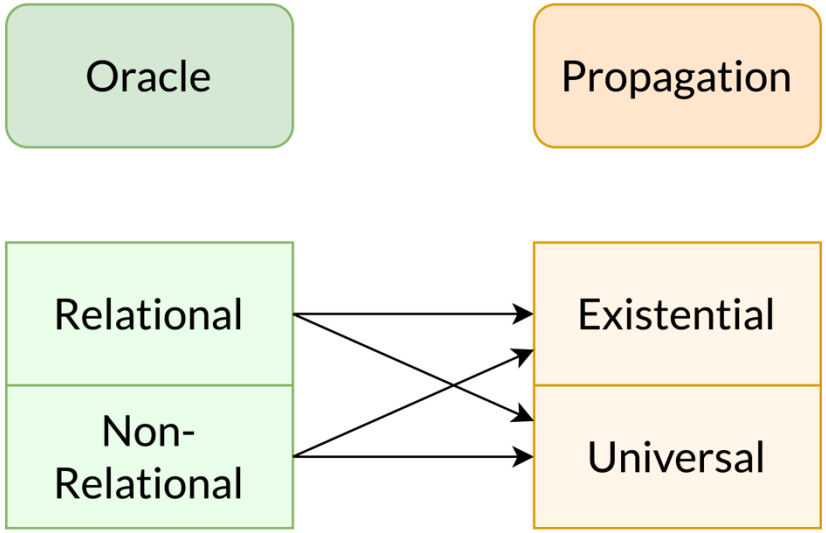
- The idea is to identify variables (Likely Unconstrained Variables) for which no information will likely be available at a given program point and propagate this lack of information
- Goal: simplify analysis, improving efficiency

LUVs' Steps

- Dataflow Analysis
  - Relational
  - Non-Relational
- Propagation
  - Existential
  - Universal
- Program Transformation

$$[\![\cdot]\!] : \mathbb{Stmt} \times \wp(\mathbb{Var}) \to \wp(\mathbb{Var})$$

Heuristics



- Four different variants
- Heuristic solutions (false pos. & false neg.)
- Final analysis is always **sound**

Non-relational oracle

$$[\![x \leftarrow ?]\!](lu) = lu \cup \{x\}$$

$$[\![x \leftarrow y \; op \; z]\!](lu) = \begin{cases} lu \setminus \{x\}, & \text{if } y \notin lu \text{ and } z \notin lu, \\ & \text{or if } op = \% \text{ and } z \notin lu, \\ & \text{or if } op = - \text{ and } y = z; \\ lu \cup \{x\}, & \text{otherwise.} \end{cases}$$

$$[\![x \leftarrow y \; op \; k]\!](lu) = \begin{cases} lu \setminus \{x\}, & \text{if } y \notin lu, \text{ or if } op = \%, \\ & \text{or if } op = * \text{ and } k = 0; \\ lu \cup \{x\}, & \text{otherwise.} \end{cases}$$

$$[\![(x \bowtie y)]\!](lu) = \begin{cases} lu \setminus \{x\}, & \text{if } y \notin lu; \\ lu \setminus \{y\}, & \text{if } x \notin lu; \\ lu, & \text{otherwise.} \end{cases}$$

Relational oracle

$$[\![x \leftarrow ?]\!](lu) = lu \cup \{x\}$$

$$[\![x \leftarrow y \; op \; z]\!]^{\mathrm{rel}}(lu) \begin{cases} lu \setminus \{x, y, z\}, & \text{if } op \in \{+, -\}; \\ [\![x \leftarrow y \; op \; z]\!](lu) & \text{otherwise.} \end{cases}$$

$$[\![x \leftarrow y \; op \; k]\!]^{\mathrm{rel}}(lu) \begin{cases} lu \setminus \{x\}, & \text{if } op = \%; \\ lu \setminus \{x, y\}, & \text{otherwise.} \end{cases}$$

False Positives - Example

$$y \leftarrow 0$$
$$x \leftarrow y * z$$

since z is LU, x is LU, even though the interval analysis would compute $x \in [0, 0]$

False Negatives - Example

$y \geq 0$ and $z \leq 0$
$x \leftarrow y + z$

x is constrained, even though the interval analysis computes $x \in [-\infty, +\infty]$

Existential vs. Universal Propagation

- Existential
  A variable is part of the LU set at the beginning of a basic block if there is at least one incoming edge along which the variable is LU

- Universal
  A variable is part of the LU set at the beginning of a basic block if it is LU along all incoming edges

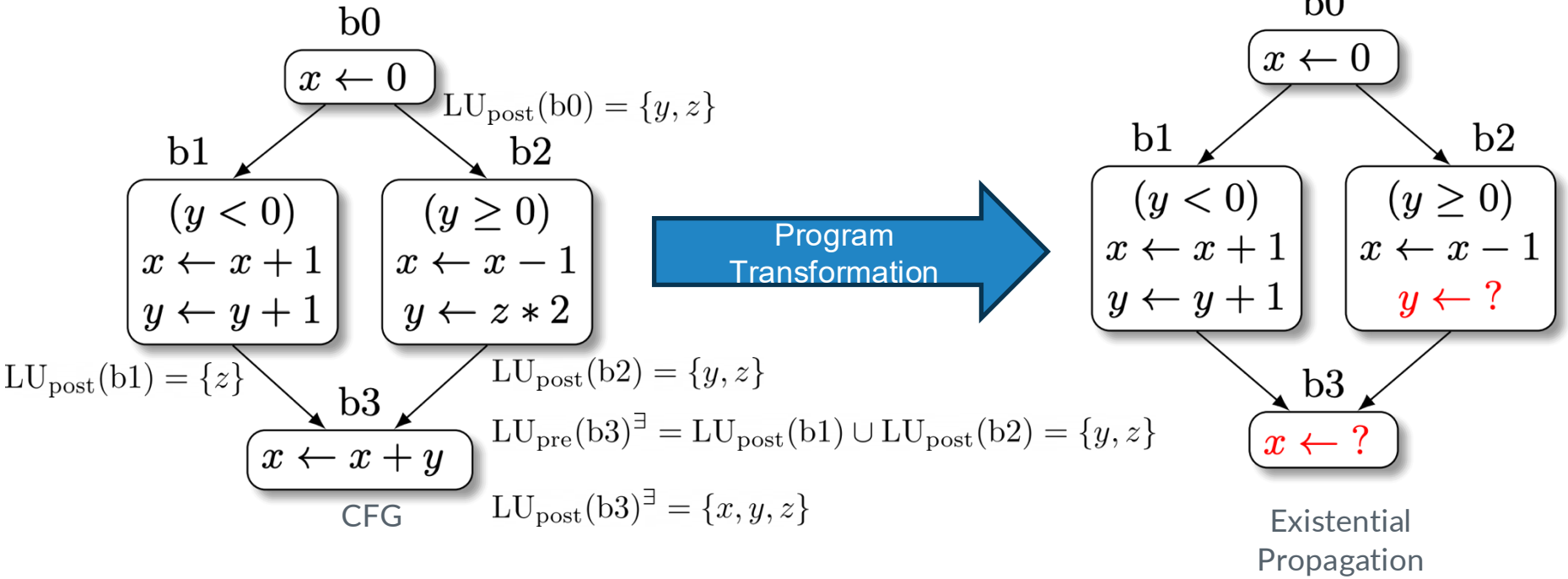
Program Transformation

Algorithm 1: Program transformation

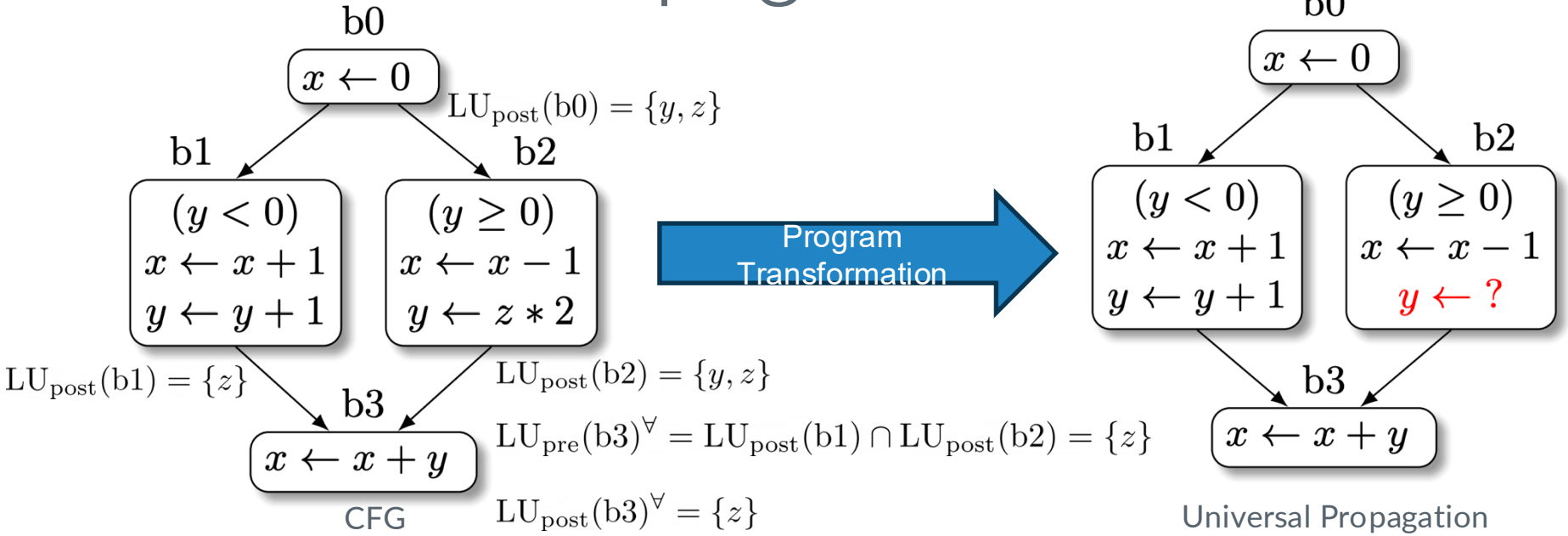
Input: $\langle N, E \rangle$ (input CFG), $\mathsf{LU}_{\mathrm{post}} : N \to \wp(\mathbb{Var})$ (LU variables map)

```
1 foreach bb ∈ N do
2 |   let lu = LU_post(bb)
3 |   foreach s = (x ← expr) ∈ bb do
4 |   |   if x ∈ lu then
5 |   |   |   replace s with s' = (x ← ?) in bb
6 |   |   end
7 |   end
8 end
```

Non-Relational Oracle + Existential Propagation



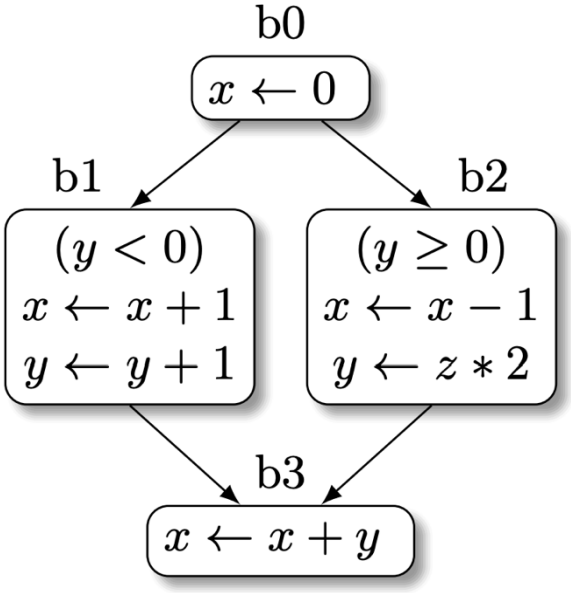
Non-Relational Oracle + Universal Propagation



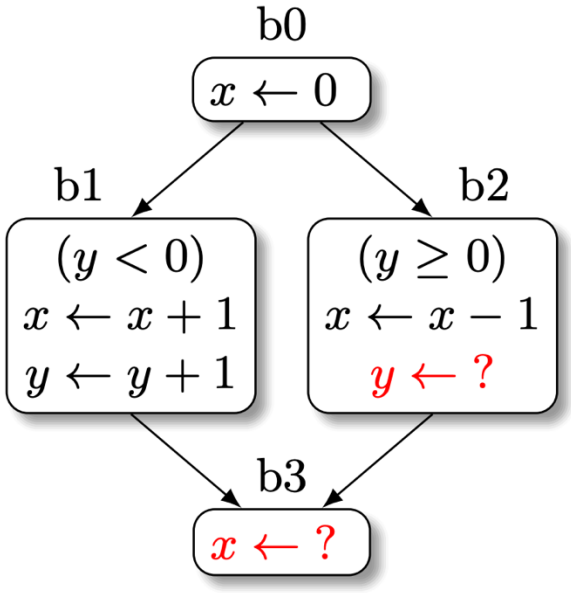
Example

$$\mathrm{LU}_{\mathrm{post}}(\mathrm{b0}) = \{y, z\}$$
$$\mathrm{LU}_{\mathrm{post}}(\mathrm{b1}) = \{z\}$$
$$\mathrm{LU}_{\mathrm{post}}(\mathrm{b2}) = \{y, z\}$$

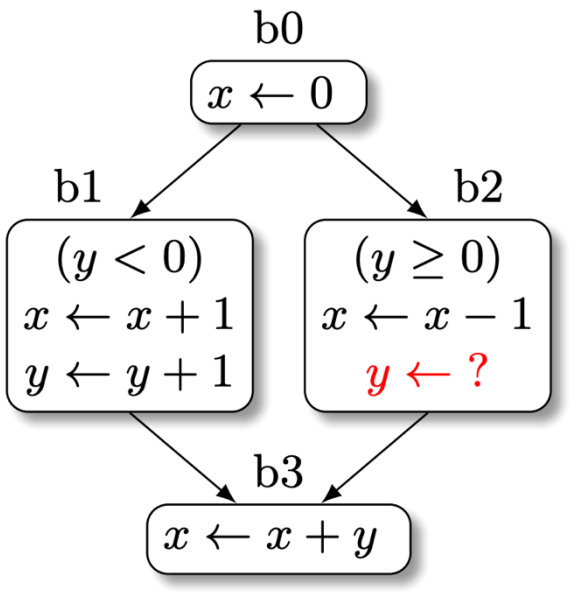
$$\mathrm{LU}_{\mathrm{pre}}(\mathrm{b3})^{\exists} = \mathrm{LU}_{\mathrm{post}}(\mathrm{b1}) \cup \mathrm{LU}_{\mathrm{post}}(\mathrm{b2}) = \{y, z\}$$
$$\rightarrow \mathrm{LU}_{\mathrm{post}}(\mathrm{b3})^{\exists} = \{x, y, z\}$$
$$\mathrm{LU}_{\mathrm{pre}}(\mathrm{b3})^{\forall} = \mathrm{LU}_{\mathrm{post}}(\mathrm{b1}) \cap \mathrm{LU}_{\mathrm{post}}(\mathrm{b2}) = \{z\}$$
$$\rightarrow \mathrm{LU}_{\mathrm{post}}(\mathrm{b3})^{\forall} = \{z\}$$



CFG



Existential Propagation



Universal Propagation

Experimental Evaluation

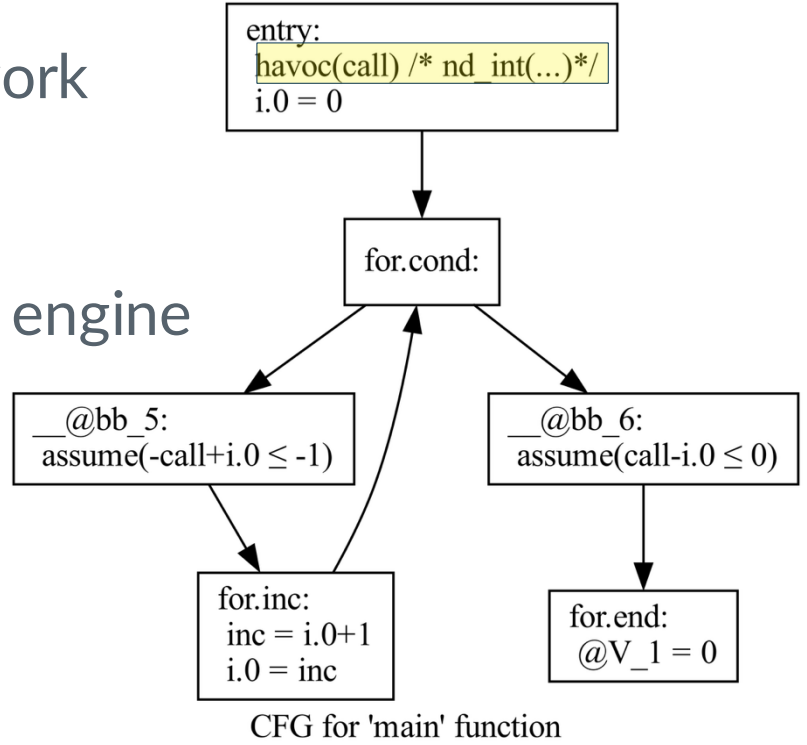
- Two benchmark suites:
  - WCET analysis (PAGAI, 34 files) [details in paper]
  - Linux drivers (SVCOMP, 10 files)

- Two domains:
  - Intervals
  - Factored Polyhedra (PPLite)

Clam/Crab

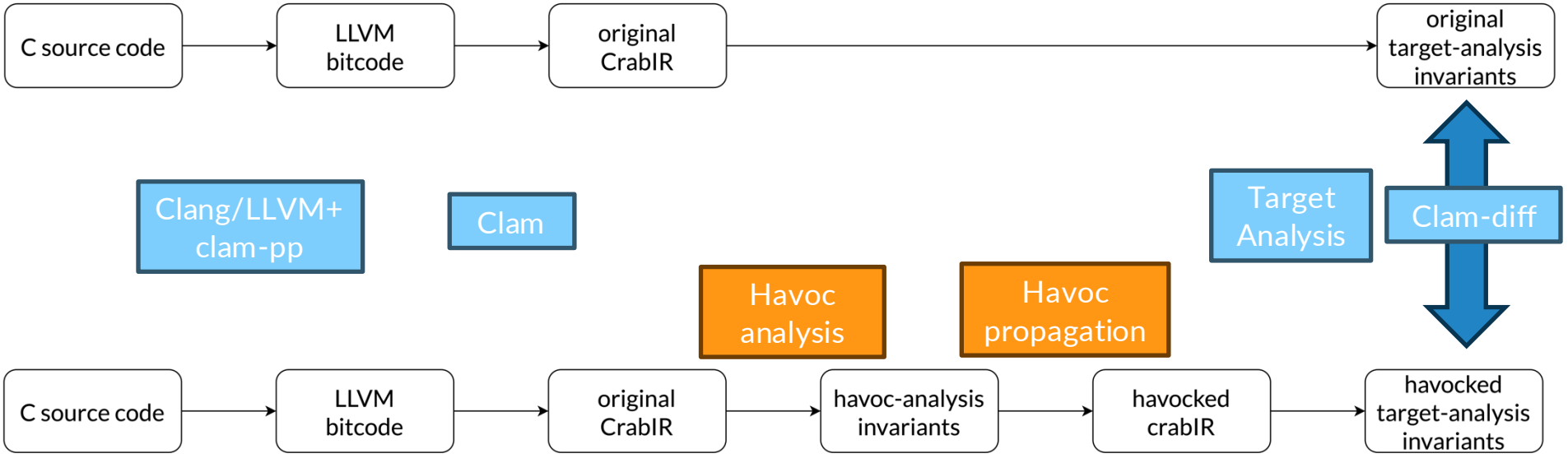
Gurfinkel, Navas; VSTTE 2021

- Modules of the SeaHorn framework
- Crab
  - C++ library for static analysis
  - Abstract domain and fixpoint engine
  - CrabIR
- Clam
  - Frontend for Crab
  - C → LLVM → CrabIR
- Clam-diff



CFG for 'main' function

Complete Pipeline



Experimental Evaluation - Stmts

stmts havocked

		non-relational				relational			
test	stmts	$LU^{\exists}$	time	$LU^{\forall}$	time	$LU^{\exists}_{rel}$	time	$LU^{\forall}_{rel}$	time
all tests	353544	119509	7014	104509	12653	14591	6771	14411	12533
wl12xx	109101	89543	332	75667	578	942	453	915	726
rtlwifi	53456	5873	1481	5873	3391	5813	1418	5813	3049
w83781d	52909	5452	2933	5450	4366	2605	2645	2605	4706
brocade	37587	4703	308	4495	666	2098	309	2039	614
libfc	31272	3108	1285	2811	2472	66	1263	65	2227
vmxnet3	19598	3983	175	3849	311	496	184	495	327
mdc	17104	2333	154	2168	314	1017	149	981	322
firewire	13196	1902	185	1701	284	800	185	747	282
solos	12465	2302	122	2301	199	732	131	730	210
abituguru	6856	310	39	194	72	22	34	21	70

$LU^{\exists}$= [4.5%; 82.1%]
$LU^{\forall}_{rel}$ = [0.2%; 10.9%]
$LU^{\exists}$ avg = 33.8%
$LU^{\forall}_{rel}$ avg = 4.1%,

Experimental Evaluation - Stmts Results

- Relational oracles are too conservative
- Non-relational oracles are rather aggressive
  - May trigger significant efficiency improvements

- Relational and non-relational variants of the havoc analysis seems to have a much greater impact than the choice between existential and universal

Experimental Evaluation – Precision

Domain: Intervals
~99% original constraints

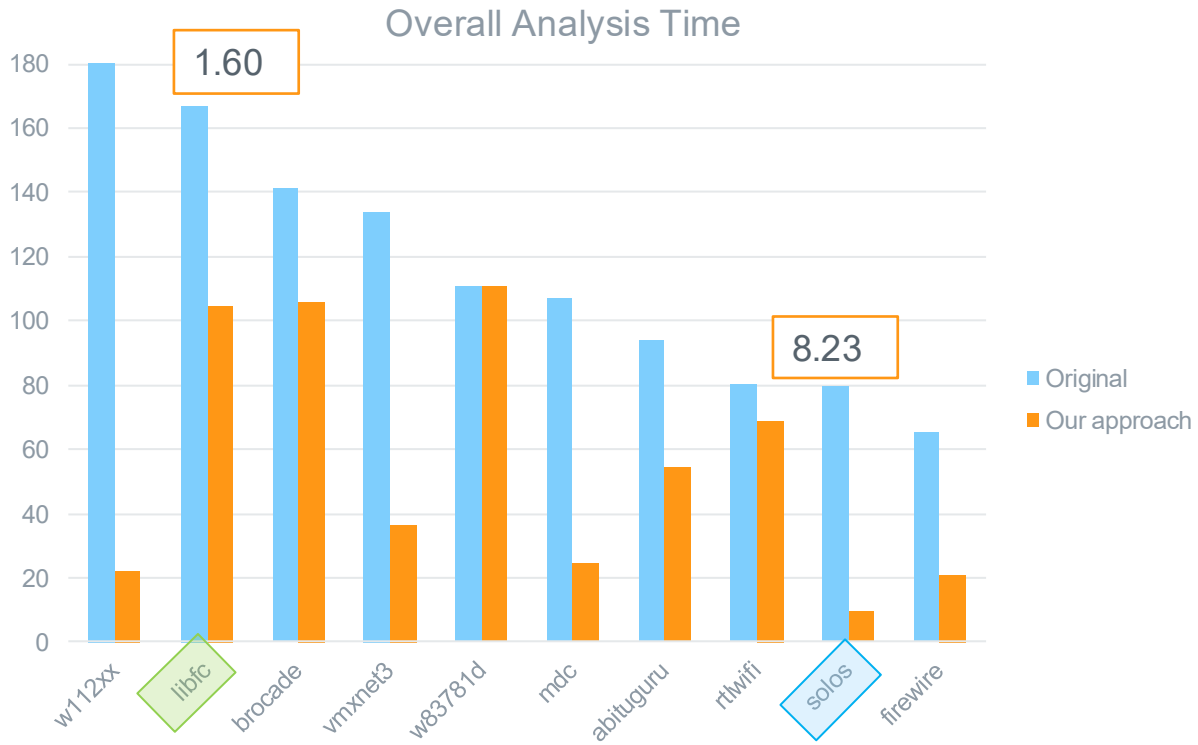
		$LU^{\exists}$ *vs* original		$LU^{\forall}$ *vs* original	
		EQ	GT	EQ	GT
# INVS	3102	2981	121	3005	97
% INVS	100.00	96.10	3.90	96.87	3.13
vmxnet3	513	416	97	416	97
mdc	112	**88**	**24**	**112**	**0**

Experimental Evaluation – Precision

		$LU^{\exists}$ *vs* original		$LU^{\forall}$ *vs* original	
		EQ	GT	EQ	GT
# INVS	3102	1389	1713	1485	1617
% INVS	100.00	44.78	55.22	47.83	53.17
vmxnet3	513	190	323	190	323
brocade	511	154	357	154	357
firewire	499	319	180	319	180
w83781d	356	95	261	95	261
solos	303	104	199	104	199
libfc	295	295	0	295	0
rtlwifi	208	5	203	5	203
abituguru	169	**91**	**78**	**168**	**1**
wl12xx	136	92	44	92	44
mdc	112	**44**	**68**	**63**	**49**

Domain: Polyhedra
~90% original constraints

Experimental Evaluation – Time (Polyhedra)



Avg. speedup 2.61

Conclusion

- Results
  - Significant speedups up to 8.23 – avg. 2.61
  - Limited loss of precision ~90% of original cons.

- Future Work
  - Further simplifications of CFG post havoc-processing
  - Improved heuristics

Unconstrained Variable Oracles for Faster Numeric Static Analyses

Vincenzo Arceri[1 [0000−0002−5150−0393]], Greta Dolcetti[1 [0000−0002−2983−9251]], and Enea Zaffanella[1 [0000−0001−6388−2053]]

Department of Mathematical, Physical and Computer Sciences
University of Parma, Parma I-43124, Italy
vincenzo.arceri@unipr.it, greta.dolcetti@studenti.unipr.it,
enea.zaffanella@unipr.it

Abstract. In the context of static analysis based on Abstract Interpretation, we propose a lightweight pre-analysis step which is meant to suggest, at each program point, which program variables are likely to be unconstrained for a specific class of numeric abstract properties. Using the outcome of this pre-analysis as an oracle, we simplify the statements of the program being analyzed by propagating this lack of information, aiming at fine-tuning the precision/efficiency trade-off of the analysis. A preliminary experimental evaluation shows that the idea underlying the

Thanks!

Unconstrained Variable Oracles for Faster Static Analyses

Vincenzo Arceri, Greta Dolcetti, Enea Zaffanella